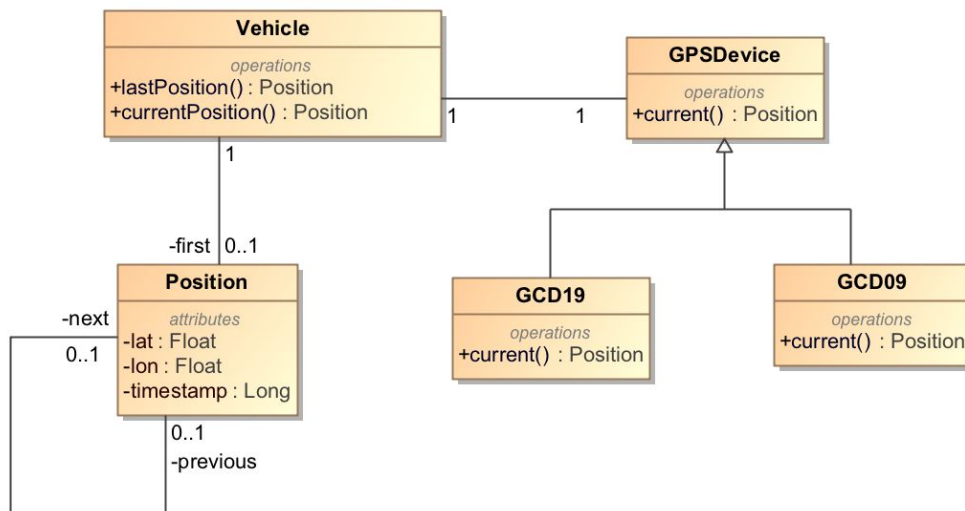


Análisis y Diseño con Patrones

PEC 2: Patrones de Diseño y Asignación de Responsabilidades

Supongamos que estamos desarrollando un software para la gestión de una flota de vehículos y que disponemos del siguiente diagrama de clases:



Como podemos ver, un vehículo tiene una serie de posiciones con una latitud, longitud y un timestamp (que representa la fecha y hora en que el vehículo estaba en la posición concreta). Las posiciones las obtenemos a partir del dispositivo GPS que tiene asociado el vehículo con la operación *current()* que cada dispositivo implementa de alguna manera que ahora no nos interesa saber.

Pregunta 1 (25%)

Supongamos ahora que queremos añadir un nuevo tipo de dispositivo (el dispositivo G200X) por el que tenemos una biblioteca de software que nos ofrece la siguiente clase para leer la posición actual:

```
class G200X{
    public Long getTimestamp()
    public float getLatitude()
    public float getLongitude()
    public float getAltitude()
}
```

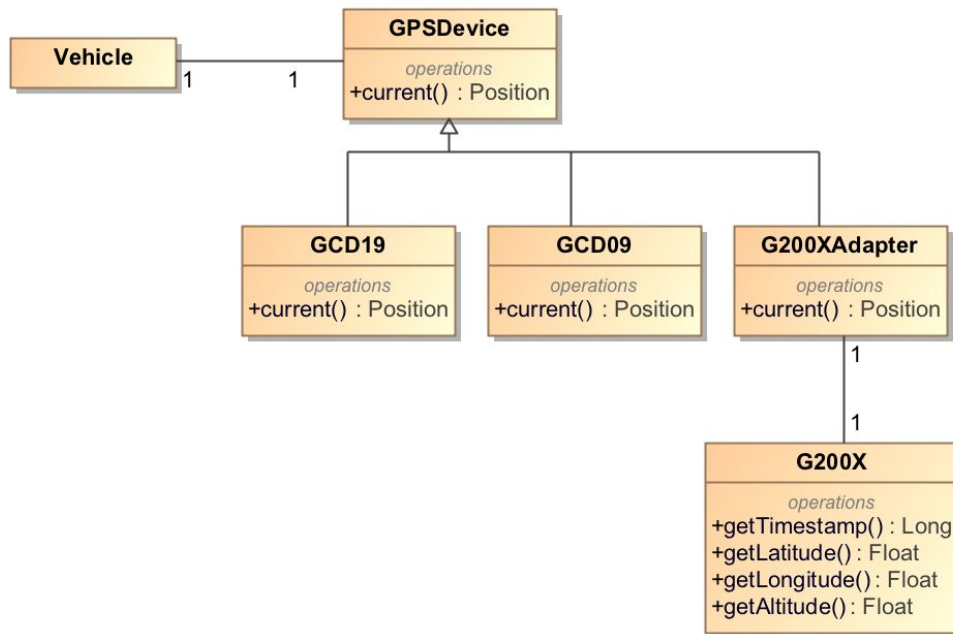
- Qué patrón de diseño recomendarías para este caso?
- Muestra el diagrama de clases resultante de la aplicación del patrón que hayas seleccionado.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación que implementa la lectura de un dispositivo G200X.

Solución

a)

En este caso podemos aplicar el **patrón Adaptador** (Adapter) ya que lo que queremos es utilizar la clase G200X utilizando un conjunto de operaciones (en concreto, la operación *current()*) diferente al que ofrece originalmente.

b)



c)

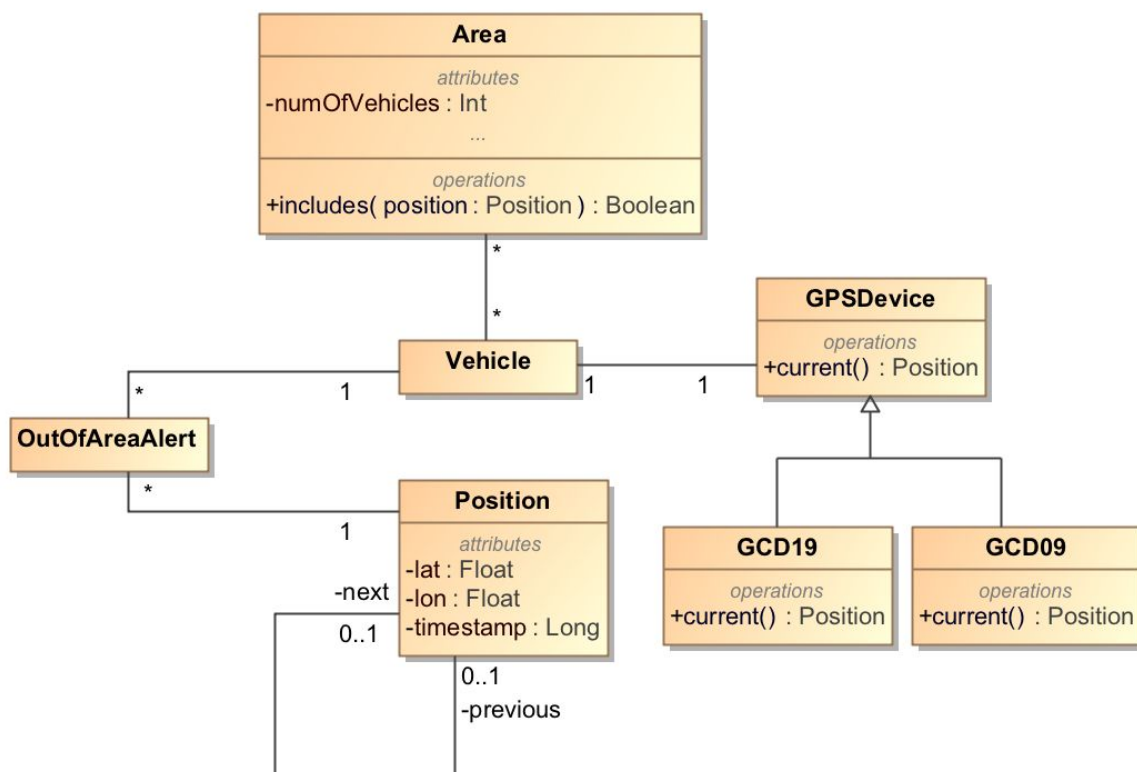
```

class G200XAdapter extends GPSDevice{
    //G200X inyectado por constructor (omitido del código para simplicidad)
    private G200X adaptee;
    override public Position current (){
        return new Position(
            lat = adaptee.getLatitude (),
            lon = adaptee.getLongitude (),
            timestamp = adaptee.getTimestamp()
        )
    }
}

```

Pregunta 2 (25%)

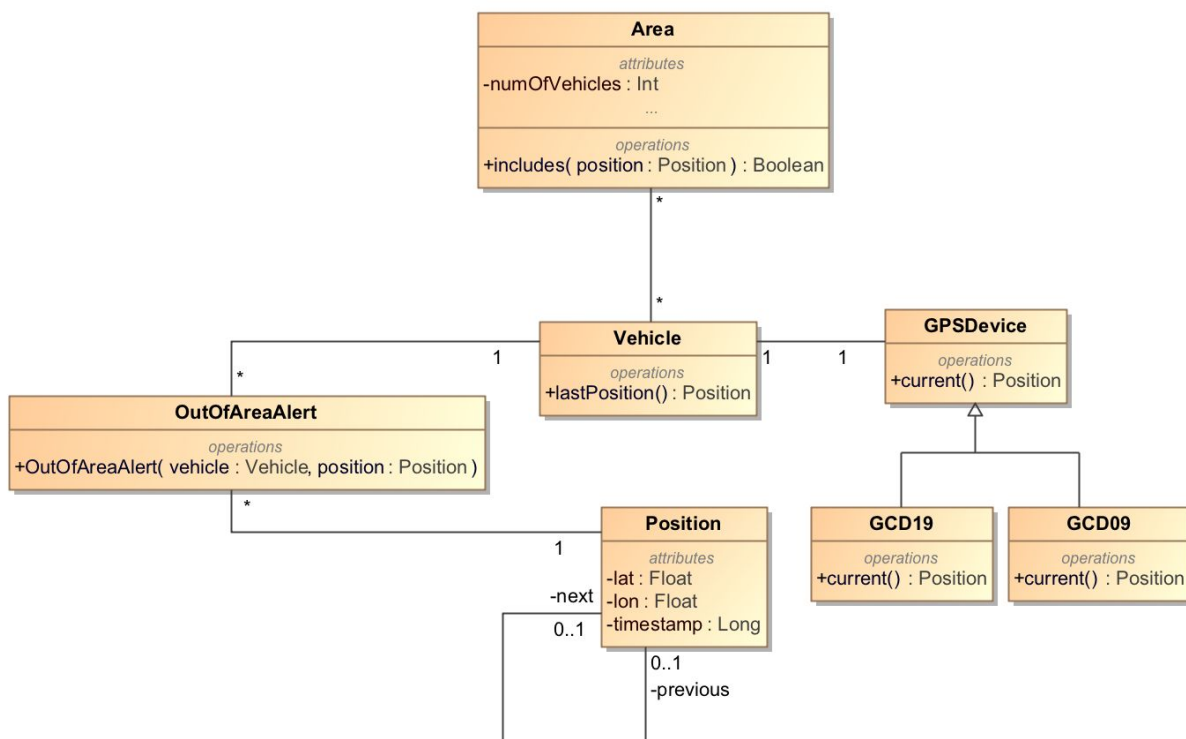
Supongamos ahora que queremos definir unas zonas geográficas que estarán asociadas a unos vehículos y que queremos monitorizar cuando estos vehículos están dentro o fuera de las áreas. Partiremos, pues, de este diagrama de clases:



La operación *includes(position: Position)* de la clase *Area* nos dice si una posición está dentro o fuera del área. Nos han dicho, también, que habrá otras funcionalidades al sistema relacionadas con el seguimiento en tiempo real de los cambios de posición de los vehículos, así que nuestra solución debe estar preparada para incluir, en el futuro, estas funcionalidades que vendrán.

En concreto, sin embargo, ahora se quiere que:

- Cuando el vehículo añade una nueva posición, se compruebe si el área del vehículo incluye la nueva posición y, en caso contrario, se genere una instancia de la clase *OutOfAreaAlert* asociada al vehículo ya la posición .
- Cuando el vehículo añade una nueva posición, si la posición anterior del vehículo estaba en el área y la nueva está fuera, se decrementa el valor del atributo *numOfVehicles* del área.
- Cuando el vehículo añade una nueva posición, si la posición anterior del vehículo estaba fuera del área y la nueva está dentro, se incremente el valor del atributo *numOfVehicles* del área.



- a) Qué patrón de diseño recomendarías para este caso?
- b) Muestra el diagrama de clases resultante de la aplicación del patrón seleccionado.
- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación que implementa la generación del instancia de *OutOfAreaAlert* (no es necesario incluir la actualización del atributo *numOfVehicles*). Puede suponer que la clase *Vehículo* ya tiene implementada la operación *lastPosition ()*: *Position* que nos devuelve la última posición del vehículo (es decir, la que acabamos de añadir) para acceder a ellos fácilmente. También que la clase *OutOfAreaAlert* tiene un constructor que recibe un *Vehículo* y una *Position* y que este constructor ya se encarga de establecer los enlaces necesarios de las asociaciones con estas clases.

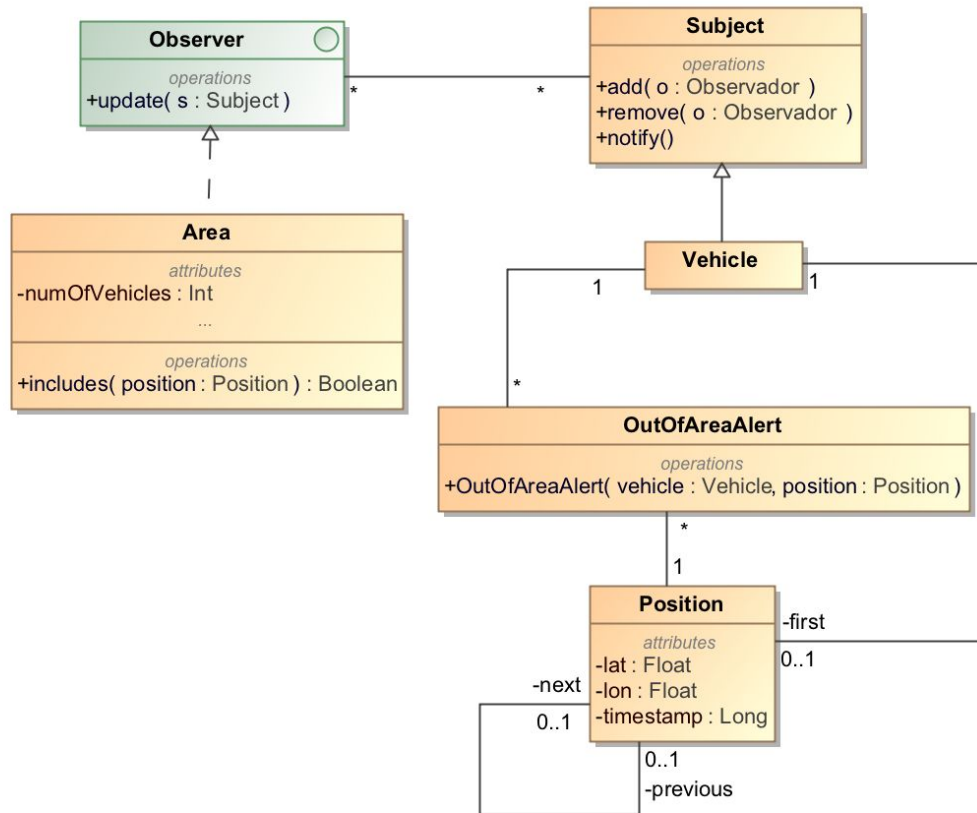
Solución

a)

Recomendaría el **patrón Observador** donde el sujeto sería el vehicle y el estado observado sería su posición. El observador sería elàrea.

Esto permitirá que, en el futuro, añadimos otros observadores que observen los cambios de posición de los vehículos tal y como se nos ha pedido.

b)



c)

```

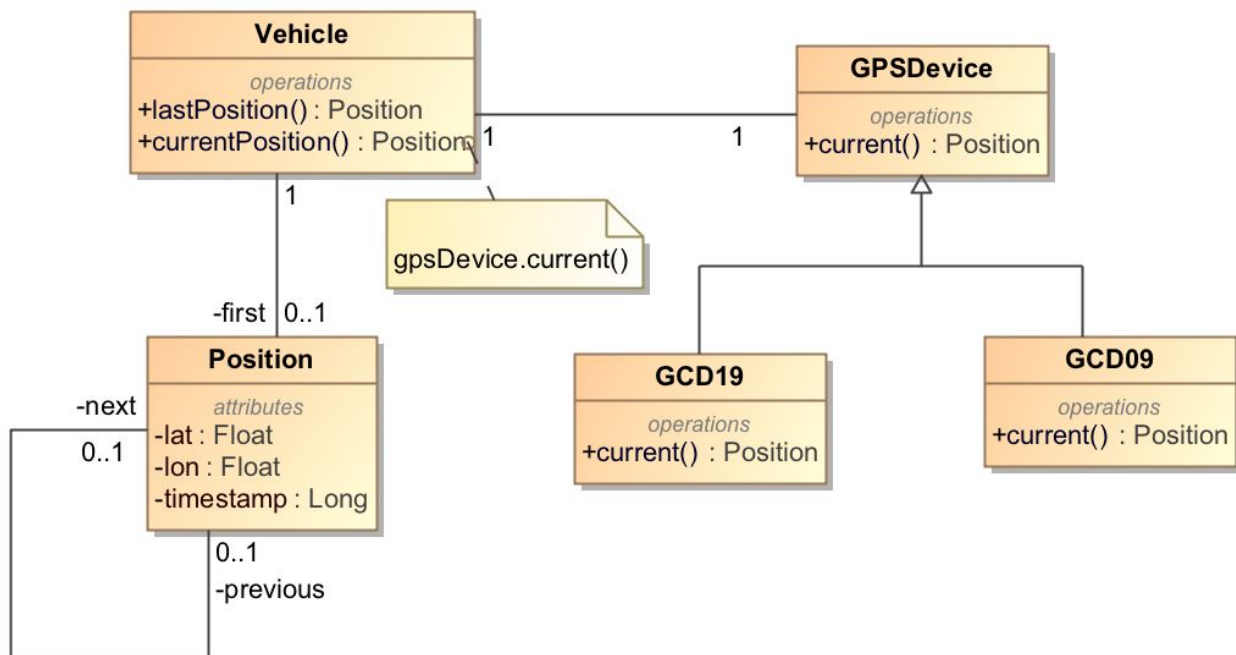
class Area implements Observer{
    override public update (subject: Subject){
        Vehículo v = (Vehículo) subject;
        Position lastPosition = v.lastPosition ();
        if (! this.includes (lastPosition)){
            new OutOfAreaAlert (v,lastPosition);
        }
    }
}

```

Pregunta 3 (25%)

Nos centraremos ahora en el dispositivo GPS. A veces puede ocurrir que no nos interese la lectura del dispositivo sino que queremos que se devuelva una posición fija. Por ejemplo, nos pasa a menudo que cuando un vehículo está en el taller, allí no tenemos buena recepción de GPS y los dispositivos generan posiciones erróneas. En estos casos, nos gustaría poder decir que el vehículo está "off-line" y poder fijar qué posición se devuelve mientras el vehículo esté "off-line". Esto también nos puede interesar, por ejemplo, si tenemos que cambiar el dispositivo GPS del vehículo (por una reparación o una actualización del mismo).

Recuerde que el diagrama de partida es este:



en concreto, queremos añadir una operación en la clase *vehículo* llamada *putOffline(position: position)* que implemente el comportamiento mencionado (suponer que el vehículo devuelve la posición indicada siempre que se invoque al operación *currentPosition():Position*).

- Qué patrón (o patrones) de diseño recomendarías para este caso (si es que recomiendas alguno)?
- Muestra el diagrama de clases resultante de añadir esta funcionalidad.

- c) Muestra el diagrama de secuencia o el pseudocódigo de la operación *putOffline* de la clase *Vehículo*.

Solución

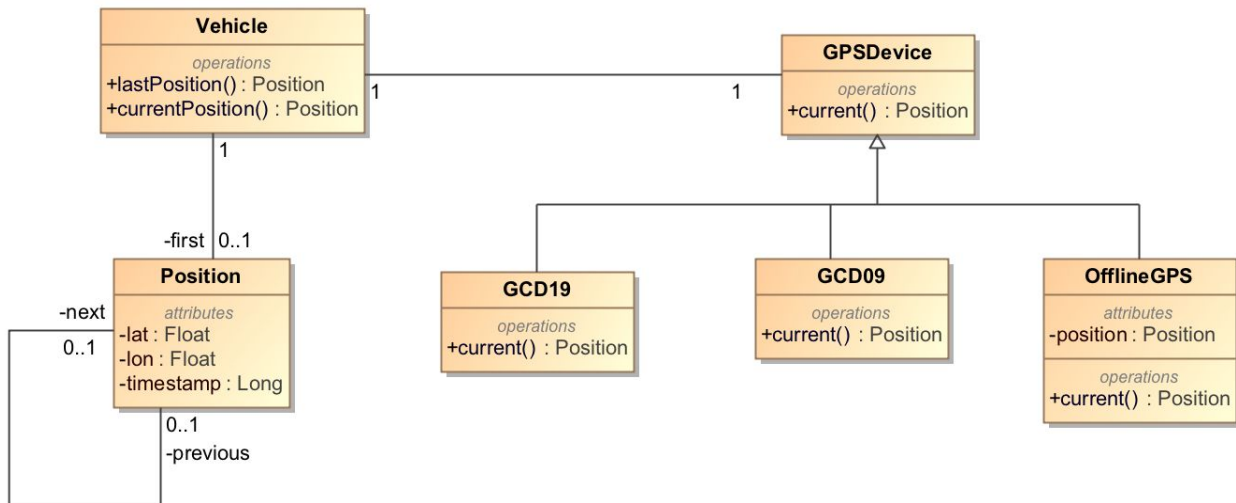
a)

Podemos aplicar el **patrón NullObject** con un nuevo tipo de dispositivo GPS llamado *OfflineGPS* que guarde una posición y que a la operación *current()* siempre devuelva esta posición.

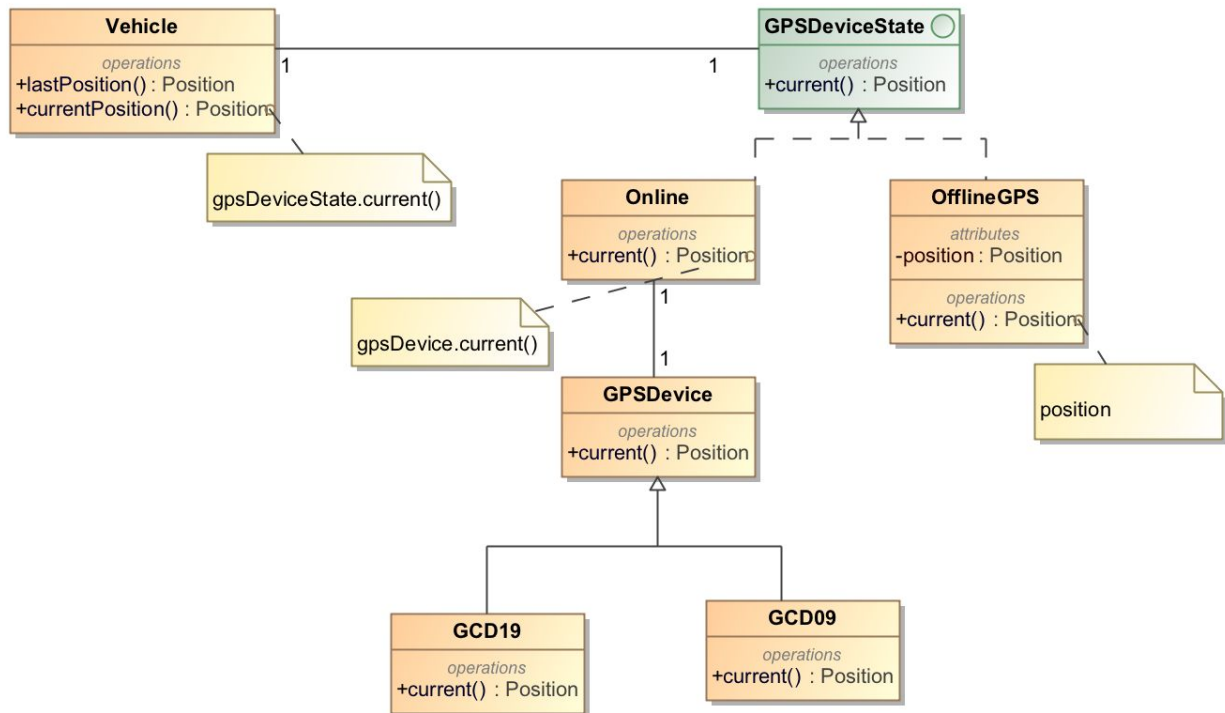
También se podía haber aplicado el **patrón Estado** a la clase *GPSDevice*, por lo que este estuviera en estado "off-line" o "on-line".

b)

Aplicar el patrón NullObject:



Aplicar el patrón Estado:



Como podemos ver, la solución con patrón Estado es más compleja que la solución con NullObject y no tiene ninguna ventaja especial, por lo que consideraremos la solución con NullObject mejor que la solución con State.

c)

```

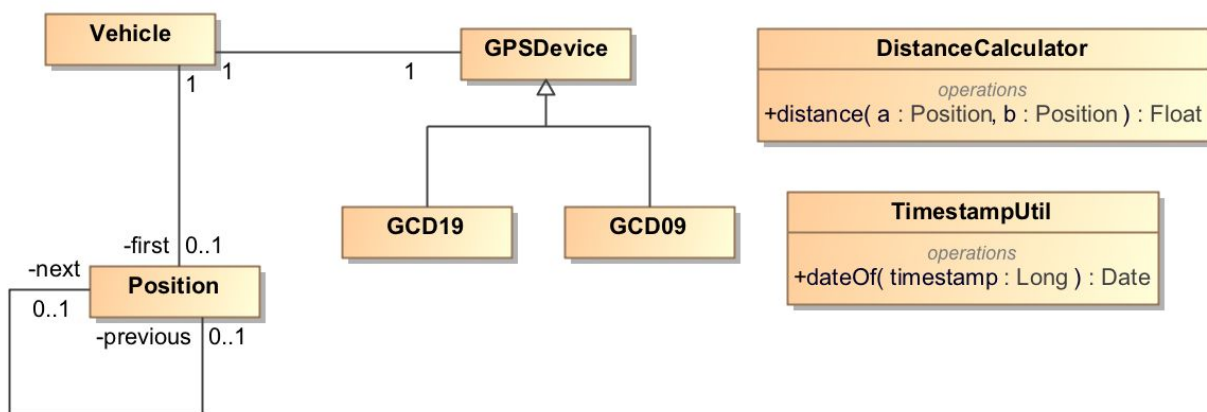
class Vehicle{
    GPSDevice gpsDevice;

    public void setOffline (position: Position){
        this.gpsDevice= new OfflineGPS (position);
    }
}

```

Pregunta 4 (25%)

Supondremos ahora que queremos calcular la distancia recorrida en kilómetros por un vehículo en una fecha concreta. Para facilitar este cálculo, supondremos que hemos aplicado el patrón de asignación de responsabilidades "Fabricación pura" y tenemos la clase *DistanceCalculator* con el método *DistanceCalculator::distance(a, b): Float* que nos calcula la distancia en kilómetros entre dos posiciones. Por otra parte, también tenemos la clase *TimestampUtil* con el método *TimestampUtil::dateOf(timestamp: Long): Date* que nos devuelve la instancia de *Date* que corresponde al timestamp



- Muestra el diagrama de secuencia o el pseudocódigo de la operación *distanceInDate(Date date):Float* de la clase *vehículo*, que devuelve la distancia recorrida en kilómetros del vehículo que invoca la operación en la fecha pasada como parámetro.
- Indica qué clase implementa las siguientes responsabilidades:
 - Determinar si el timestamp corresponde a la fecha pedida (es decir, compara el long de *timestamp* con el *date* de *distanceInDate*).
 - Determinar la posición anterior a una posición concreta
 - Determinar si una distancia debe sumarse o no
 - Sumar las distancias

Solució

a)

```
class Vehicle{
    private Position first;
    public Float distanceInDate (Date date){
        Float result = 0;
        Position firstInDate = findFirstPositionInDate (date);
        Position current = firstInDate.next;
        while (current! = null and
            TimestampRange.belongsToDate (date,current.timestamp)){
            result = result + current.distanceFromPrevious();
            current = current.next;
        }
    }
    Private Position findFirstPositionInDate(Date date){
        Position current = first;
        while (current! = null and
            TimestampUtil.dateOf (current.timestamp)! = date){
            current = current.next;
        }
        Return current;
    }
}
```

```
Class Position{  
    Float lat;  
    Float lon;  
    Long timestamp;  
    Position previous;  
    Position next;  
    public Float distanceFromPrevious (){  
        if(previous == null){  
            return 0;  
        } else {  
            return DistanceCalculator.distance (this, previous);  
        }  
    }  
}
```

B)

- Determinar si el timestamp corresponde a la fecha pedida (es decir, compara el long de *Timestamp* con el *date* de *distanceInDate*): Vehículo
- Determinar la posición anterior a una posición concreta: Position
- Determinar si una distancia se debe sumar o no: Vehículo
- sumar las distancias: Vehículo